

How AI Code Review Works

Understand the technologies behind automated code review and where they still need a human in the loop.

For developers and team leads who want to understand what AI code review tools actually check and where they fall short · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ AI code review layers several techniques: static analysis (before running code), dynamic analysis (while it runs), rule-based linting, and LLMs that add contextual understanding on top.
- ✓ Static analysis parses source text for syntax errors, style violations, and known-bad patterns without ever executing the code.
- ✓ Dynamic analysis, including DAST (Dynamic Application Security Testing), tests the running application by sending real inputs and observing behavior, catching runtime issues static analysis cannot see.
- ✓ Rule-based linters enforce fixed, deterministic style and safety rules; LLMs go further by recognizing patterns and intent that no fixed rule set was written to cover.
- ✓ Consistency is a structural benefit: an AI reviewer applies the same standard to every PR, where two human reviewers can reasonably disagree on what to prioritize.
- ✓ AI review is not free of error: it produces false positives (flagging non-issues) and false negatives (missing real vulnerabilities), so it augments human review rather than replacing it.
- ✓ Context engineering, structuring what information the model receives about your codebase and standards, directly determines how relevant its suggestions are.
- ✓ Successful adoption requires encoding team standards somewhere the AI can read them, integrating the tool into the existing PR workflow, and tracking whether it actually improves defect rates and review turnaround, not just adding noise.

What it covers

- 01 The pull-request bottleneck AI code review is meant to fix** 0:00
Slow, inconsistent human review creates a compounding cost: the longer a PR waits, the more it drifts from main, and the friction spreads across the whole pipeline from PR to production.
- 02 What AI code review actually is** 0:44
AI code review is software that uses machine learning and large language models to scan code changes, detect problems such as security flaws or style violations, and often propose a fix, working alongside (not instead of) human reviewers.
- 03 Why teams adopt it: consistency, learning, and less technical debt** 2:13
Beyond speed, AI review applies the same standard on every PR, turns feedback into a learning moment for the author, and catches problems while they are still cheap to fix.
- 04 Static code analysis: catching problems before code runs** 3:40

Static analysis inspects source code without executing it, looking for syntax errors, security risks, and violations of coding standards before the code ever ships.

05 **Dynamic code analysis and DAST: testing the running app** 4:29

Dynamic analysis tests software while it executes, uncovering runtime vulnerabilities and performance bottlenecks that static analysis cannot see because they only appear when the code actually runs.

06 **Rule-based systems, linters, and where LLMs add reasoning** 5:16

Linters and other rule-based systems enforce fixed coding guidelines deterministically; large language models go further because they are trained on enough code, documentation, and discussion to recognize context and meaning, not just pattern matches against a fixed rule list.

07 **Reaching beyond the codebase: tools, docs, and context engineering** 6:44

Modern AI review systems connect to developer environments, testing frameworks, documentation, and even the live web, rather than relying only on what the model learned during training, making the analysis more current and more aware of a project's actual context.

08 **Where AI code review breaks down** 7:24

Three real limitations: developers can over-rely on automated feedback and stop thinking about architecture, the model may lack project-specific context, and it will produce both false positives and false negatives.

09 **Adopting AI code review without losing human judgment** 8:09

Rolling it out well means choosing a tool that fits your stack, encoding your standards somewhere the AI can read them, integrating into the existing PR workflow, tracking whether it actually improves quality signals, and keeping a human as the final reviewer throughout.

Glossary

AI code review — The use of machine learning and large language models to automatically scan code changes, detect problems, and often suggest fixes, alongside human reviewers.

Dynamic code analysis — Testing software while it is actually running, to uncover runtime issues like performance bottlenecks and race conditions that static analysis cannot see.

Rule-based system — A tool, such as a linter, that enforces a fixed, explicit set of coding rules deterministically, catching only what a rule was written to detect.

Context engineering — Deliberately structuring the information (standards, docs, related code) given to a model so its output is relevant and accurate to a specific project.

False negative — A case where a review tool fails to flag a real issue, creating false confidence that the code is safe.

Static code analysis — Inspecting source code for syntax errors, security risks, and style violations without executing it.

DAST (Dynamic Application Security Testing) — A form of dynamic analysis that simulates real-world attacks against a running application by sending crafted inputs and logging unusual or insecure responses.

Large language model (LLM) — A model trained on large amounts of text, including code, documentation, and discussion, that can recognize contextual and semantic patterns beyond fixed rules.

False positive — A case where a review tool flags code as a problem when it is not actually one, wasting reviewer time.

Technical debt — The accumulated cost of shortcuts and unresolved issues in a codebase, which grows more expensive to fix the longer it goes unaddressed.

Explanations are AI-generated. Check anything you intend to rely on.